

GANADI: Uncovering C/C++ OSS Reuse Genealogies via Pivotal Function-Based Clustering to Enhance Supply Chain Security

Dongyeon Kim
Korea University
Seoul, Republic of Korea
kimdongyeon@korea.ac.kr

Seunghoon Woo✉
Korea University
Seoul, Republic of Korea
seunghoonwoo@korea.ac.kr

Heejo Lee✉
Korea University
Seoul, Republic of Korea
heejo@korea.ac.kr

Abstract

We present Ganadi, a systematic approach for identifying C/C++ OSS reuse genealogies to enhance software supply chain security. Understanding OSS reuse genealogy is crucial for improving SBOM completeness and prioritizing security remediation across supply chains. Although existing approaches can identify reused components and vulnerabilities within a project, they fail to trace OSS reuse paths through intermediate projects, limiting their effectiveness in securing supply chain ecosystems. To address this limitation, Ganadi constructs reuse genealogies by clustering downstream projects based on shared characteristics of origin-derived code (called pivotal functions), and then inferring reuse direction among the projects within each cluster. When applied to 20 widely reused OSS projects with over 1,500 propagation paths, Ganadi achieved 84.85% precision and 95.76% recall in identifying reuse genealogies, outperforming existing approaches that achieved at most 23.21% recall. Leveraging OSS reuse genealogy for vulnerability detection, we identified 48 unpatched vulnerabilities in real-world popular C/C++ projects. Among them, 23 were patched following our responsible disclosure (including one CVE ID assigned), demonstrating the practical impact of genealogy-based vulnerability management.

CCS Concepts

• **Security and privacy → Software security engineering**.

Keywords

Software Reuse Genealogy; Supply Chain Security; Vulnerability Management.

ACM Reference Format:
Dongyeon Kim, Seunghoon Woo, and Heejo Lee. 2026. GANADI: Uncovering C/C++ OSS Reuse Genealogies via Pivotal Function-Based Clustering to Enhance Supply Chain Security. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3832783.3837498

1 Introduction

Open-source software (OSS) is widely reused in modern software development, accelerating development cycles and reducing implementation costs. However, this widespread reuse poses security risks that are often overlooked by existing approaches.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3837498

When developers copy-paste, fork, or adapt OSS components, they create hidden propagation paths through which vulnerabilities can spread across projects, yet these reuse relationships are rarely documented or tracked.

Although a Software Bill of Materials (SBOM) documents which OSS components are used in a project [1, 25, 31], prior SBOM-based approaches do not fully capture supply chain security. Effective security analysis requires understanding not just *what* components are used, but *how* they are propagated: through which intermediate projects and with what modifications. Accordingly, recent SBOM standard updates [3] have strengthened requirements for documenting dependency relationships.

We address this gap by reconstructing the *reuse genealogy*: the origin-to-descendant relationships that reveal how code transitively flows through the OSS ecosystem. This not only enhances SBOM completeness, but also enables understanding the propagation paths and scope of vulnerabilities in complex supply chain ecosystems, thereby facilitating more effective vulnerability management (Section 4.3). For example, if a project reuses the `zlib` compression library, it is essential to determine whether the code was reused directly from the origin `zlib` or through intermediate projects that may have modified it. Tracing this path is critical for security management, as it reveals how vulnerabilities spread [10, 22, 34], how licenses are modified [4, 30, 36], and how code changes accumulate across intermediate projects (Section 2.3).

To our knowledge, no previous work has focused on identifying C/C++ OSS reuse genealogies. Even when developing new techniques, the problem remains non-trivial due to two technical challenges (see Section 2.2). First, *lack of explicit reuse metadata.* C/C++ projects typically reuse code through direct copying without declaring dependencies or maintaining reuse records. Consequently, there are no explicit indicators to identify what code was reused or how it propagated [14, 35]. Second, *difficulty in determining reuse direction.* Even when available metadata is used to infer reuse genealogy, determining the direction of reuse remains challenging. Common metadata features, such as commit timestamps, are often inconsistent and easily manipulated during reuse [29, 33, 34].

Limitation of existing approaches. Existing software composition analysis (SCA) approaches (*e.g.*, [5, 14, 35, 37]) have focused on identifying reused components and inferring their origins, without considering the reuse genealogy. For example, Centris [35], TPLite [14], and BinaryAI [13] attempt to identify components using function birth time, metadata, and function similarity, respectively. Because they fail to capture the directionality of reuse, they cannot resolve issues that emerge from the absence of reuse genealogy analysis, such as hidden vulnerability propagation. Existing propagated vulnerability detection approaches (*e.g.*, [6, 20, 32, 33])

cannot track transitive vulnerability propagation across the broader supply chain or systematically identify all affected downstream projects, limiting their applicability to supply chain-wide security management. Several studies attempt to track software evolution (*e.g.*,[8, 11, 24]), but they focus on changes within a single project; thus, they do not capture reuse genealogy across multiple projects.

To address these limitations, we present Ganadi (Genealogy ANAlysis for Dependency Inference), a new approach for identifying OSS reuse genealogies. Ganadi uses *pivotal functions* (*i.e.*, functions in downstream files that contain at least one function identical to one in the origin OSS) to focus on reused code and exclude irrelevant code. It then performs (1) *software clustering*, (2) *reuse inference*, and (3) *graph construction*.

Approach overview. Ganadi begins by constructing a pool of popular C/C++ software projects. Given an OSS (origin), Ganadi identifies candidates within the pool that share identical functions with the origin. For each candidate, Ganadi identifies pivotal functions from files containing origin-derived code (Section 3.2.3). It then performs software clustering of candidates based on shared characteristics (*e.g.*, file paths). This effectively groups related projects and filters out unrelated software (Section 3.2.4).

Within each group, Ganadi examines pairwise reuse relationships and aggregates them to construct the consolidated genealogy (Section 3.3). Here, Ganadi identifies reuse relationships by jointly analyzing code similarity and reuse directionality, leveraging both explicit (*e.g.*, fork) and implicit evidence (*e.g.*, code birth dates). Finally, Ganadi constructs a *reuse genealogy graph* (Section 3.4) by connecting identified reuse relationships from all clusters to the origin, following priority-based rules to minimize false alarms.

Evaluation. We constructed a software pool of 2,500 popular C/C++ projects from GitHub, and applied Ganadi to 20 widely reused OSS projects as origins to identify their reuse genealogies within the pool. Ganadi identified over 1,500 reuse relationships with 84.85% precision and 95.76% recall, outperforming existing approaches that achieved at most 23.21% recall due to their limited ability to capture transitive reuse propagation (Section 4.1). To evaluate Ganadi from a supply-chain security perspective, we incorporated reuse genealogy into a vulnerability management process. Across 20 widely reused OSS projects, incorporating genealogy into existing detectors [6, 20] improved precision by 13.7%, maintained comparable F1-scores using only 11% of the baseline dataset, and clarified vulnerability propagation during manual analysis (Section 4.3). Using Ganadi, we identified 48 unpatched vulnerabilities in real-world projects, 23 of which were patched following our disclosure.

Contributions. We summarize our contributions below.

- We identify the importance of OSS reuse genealogy analysis, and propose Ganadi, a novel approach to identify C/C++ OSS reuse genealogies to enhance supply chain security.
- We introduce a pivotal function-based clustering approach that enables accurate reuse relationship identification even in challenging scenarios with many intermediate projects.
- Ganadi identifies reuse genealogies with 84.85% precision and 95.76% recall, outperforming prior approaches, and uncovers 48 unpatched vulnerabilities through genealogy-driven vulnerability discovery.

2 Motivation

We introduce basic terms and the problem with its technical challenges, then motivate our approach through a concrete example.

2.1 Basic Terms

We define three basic terms: OSS components, OSS reuse, and reuse relationships. *OSS components* represent a whole or part of OSS that can be independently reused in other software [23, 35]. *OSS reuse* refers to the utilization of OSS components within the target software. *reuse relationship* refers to the link between two software projects in which one project reuses code from the other.

2.2 Problem Overview and Challenges

Recent software projects are composed of a mix of proprietary code and reused components [14, 35]. Let $F(X) = C(X) \cup D(X)$ denote the set of functions in project X, where $C(X)$ is the set of reused functions and $D(X)$ is the set of self-developed functions.

Given a target software X, the traditional goal of SCA approaches (*e.g.*, [13, 35, 37]) is to identify $C(X)$ and the corresponding *origin projects* from which each function in $C(X)$ was reused.

In contrast, Ganadi aims to identify all software projects that have reused a given origin project o (directly or transitively), as well as the propagation paths of this reuse. Let $\mathcal{R}(o)$ denote the set of all projects that reused o, where P denotes a software project: $\mathcal{R}(o) = \{P \mid C(P) \text{ contains functions derived from } o\}$.

Our goal is to identify the reuse genealogy by constructing a directed acyclic graph $G_o = (V, E)$ where $V = \mathcal{R}(o) \cup \{o\}$ and $E = \{(X, Y) \mid Y \text{ directly reuses code from } X\}$.

Technical challenges. However, addressing this problem is a non-trivial task mainly due to the following two challenges. First, *the lack of explicit linkage in C/C++ code reuse.* Although package managers for C/C++ such as `Conan` exist, code reuse primarily occurs via direct copy-and-paste [35]. As a result, reused components do not leave clear structural or metadata traces, making it difficult to detect reuse relationships through conventional static or dependency analysis.

Second, *the difficulty of determining reuse direction.* Even when relying on timestamp-based metadata such as commit dates [34], inferring the direction or path of reuse remains challenging, especially when the origin is reused in parallel by multiple projects. For example, if project A is reused by both B and D, which are later reused by C and E, respectively, the two reuse paths $A \rightarrow B \rightarrow C$ and $A \rightarrow D \rightarrow E$ become interleaved in time. In such cases, the fact that B's reuse occurred before D's does not imply a reuse path from B to D. Other features (*e.g.*, commit messages or file structures) are often noisy, inconsistent, or missing in the OSS ecosystem [29, 34].

2.3 Motivating Example

Without a clear understanding of OSS reuse genealogy, it is challenging to address security issues throughout the software supply chain. Suppose we identify the OSS components used in `Redis` and `Dragonfly`, and observe that both include `Lua`. Existing SCA techniques, which rely on component names and versions, attempt to detect known vulnerabilities by checking whether the version of `Lua` used in each project is affected [21, 32, 35], referencing public vulnerability databases (*e.g.*, NVD).

Listing 1: A patch snippet for CVE-2020-14147 (ef764d). Neither this code nor the patch exists in the upstream Lua.

```
1 //Path: Redis/deps/lua/src/lua_struct.c
2 - static int getnum (const char **fmt, int df) {
3 + static int getnum (lua_State *L, const char **fmt, int df) {
4     ...
5     int a = 0;
6     do {
7 +     if (a > (INT_MAX / 10) || a * 10 > (INT_MAX - (**fmt - '0')))
8 +       luaL_error(L, "integral size overflow");
9       a = a*10 + *((*fmt)++) - '0';
```

Listing 2: A vulnerable code snippet found in the modified Lua within Redis reused by Dragonfly (now patched following our disclosure).

```
1 //Path: Dragonfly/src/redis/lua/struct/lua_struct.c
2 static int getnum (const char **fmt, int df) {
3     ...
4     int a = 0;
5     do {
6       a = a*10 + *((*fmt)++) - '0';
```

However, Dragonfly reuses Redis, and Redis does not use the upstream Lua as-is; it incorporates and modifies certain parts of the Lua codebase. For example, CVE-2020-14147 was discovered by Redis in their modified Lua component. The vulnerability resides in lua_struct.c, which does not exist in the upstream Lua. This integer overflow vulnerability was mitigated by validating the range of values assigned to the vulnerable variable a (Listing 1).

If the reuse genealogy Lua → Redis → Dragonfly is not identified, existing SCA-based approaches miss the fact that Dragonfly inherited a vulnerability introduced by Redis, which is not present in the original Lua (Listing 2). This leads to blind spots in vulnerability detection and hampers timely patching in downstream projects. Even fingerprint-based SCA tools (*e.g.*, [37]) fail here, as lua_struct.c does not exist in upstream Lua and thus no reference fingerprint exists for it. Note that clone-based vulnerability detection approaches (*e.g.*, Movery [33], FIRE [6]) can detect this vulnerability, as the reused code remains largely unmodified. However, Ganadi complements rather than replaces them: they identify point-in-time code similarity, whereas Ganadi reconstructs propagation paths across intermediate projects, reducing manual analysis overhead (see Section 4.3). This vulnerability remained unpatched until June 2025 and was subsequently fixed through our report.

3 Design of Ganadi

3.1 Overview

Figure 1 illustrates the high-level workflow of Ganadi, which comprises three phases: *software clustering* (P1), *reuse inference* (P2), and *graph construction* (P3). Given an OSS project (an origin), in P1, Ganadi identifies candidates within a pool of popular software projects and organizes them into clusters based on shared characteristics (*e.g.*, reused file paths). In P2, Ganadi analyzes pairwise reuse relationships within each cluster using pivotal functions to measure similarity and determine reuse direction. In P3, Ganadi combines all relationships to construct a reuse genealogy graph, revealing propagation paths.

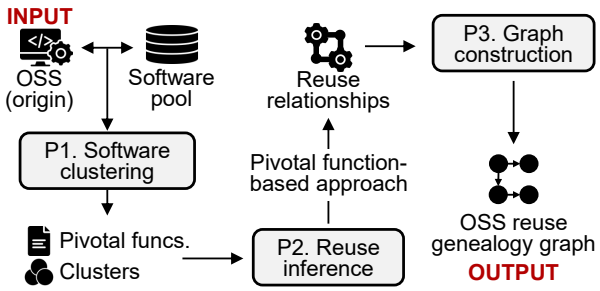


Figure 1: High-level overview of Ganadi.

3.2 Software Clustering (P1)

3.2.1 Software pool construction. Ganadi first constructs a software pool from which the origin may have been reused, to identify its reuse genealogy. To this end, Ganadi collects function codes and metadata from popular software projects. Section 4 presents a detailed explanation of the software pool implementation.

- ***Function code.*** Ganadi collects the code of all functions belonging to each software. This is the primary basis for measuring code similarity between software projects.
- ***Metadata of functions.*** Ganadi extracts two metadata elements for each function: (1) the release date of the version in which the function first appeared and (2) the associated file path. This metadata plays a crucial role in reuse analysis.

3.2.2 Candidate identification. Based on the established software pool, Ganadi identifies candidate projects that may exhibit reuse relationships with the origin. To this end, Ganadi focuses on functions shared between the origin and each project in the pool.

To ensure robustness against changes that do not affect code semantics, Ganadi applies normalization to all functions contained in a software project, following practices from prior work (*e.g.*, [20, 33, 39]). This includes removing whitespace, line breaks, and comments, as well as converting all characters to lowercase.

Note that Ganadi operates on raw source code prior to compiler preprocessing. Macro definitions and conditionally compiled blocks (*e.g.*, #ifdef) are treated as literal text during normalization and matching. Consequently, differences introduced at compile time, such as macro expansion or compiler flags, do not affect our comparison, which depends only on the normalized source text.

Ganadi then compares all normalized functions from the origin against those from each software function set in the software pool. To improve candidate detection accuracy, Ganadi extracts all functions from all versions of the origin that follow semantic versioning (*i.e.*, major.minor.patch).

Subsequently, Ganadi identifies syntactically *identical* functions via exact string matching on normalized code, based on the principle that reuse relationships manifest through at least one identical function. Only projects sharing at least θ of the origin's functions are considered candidates, where θ is set to a low value (*e.g.*, 1%) to capture even cases where only a small portion of the origin (*e.g.*, only several functions) is reused. From these candidates, Ganadi derives the code regions on which the remaining analysis focuses.

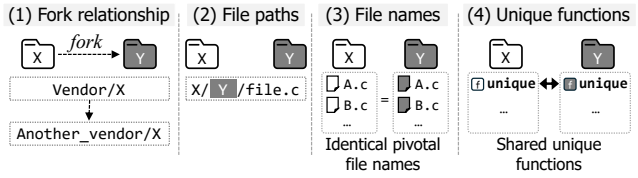


Figure 2: Four criteria for clustering. If two candidates X and Y satisfy any criteria, they are grouped into the same cluster.

3.2.3 Pivotal files and functions. We introduce two key concepts:

- **Pivotal files.** We define a *pivotal file* as a file containing at least one function identical to that of the origin.
- **Pivotal functions.** We define a *pivotal function* as a function contained in a pivotal file.

Existing SCA approaches compare entire codebases against the origin, which is not only inefficient but also introduces noise from unrelated code. Ganadi instead introduces pivotal functions, which confine the analysis to code regions derived from the origin; this scoping enables accurate pairwise comparison even among projects with large, heterogeneous codebases.

Let X denote a software project, and L_X represent the set of pivotal files within X. We define $\mathcal{F}(l)$ as the set of functions contained in the file l. The set of all *pivotal functions* of X (F_X) is given by:

$$F_X = \bigcup_{l \in L_X} \mathcal{F}(l)$$

Based on the identical functions identified in Section 3.2.2, Ganadi extracts pivotal files and functions for each candidate.

3.2.4 Multi-criteria clustering. Ganadi then performs clustering based on the pivotal files and functions. This enables Ganadi to isolate unrelated candidates and focus reuse analysis on groups of related projects, thereby reducing false alarms.

For each pair of candidate projects (X, Y), Ganadi examines the following four characteristics.

(1) ***Fork relationships.*** If either X is a fork of Y or *vice versa*, they are grouped into the same cluster.
(2) ***File paths.*** X and Y are clustered together if either project's name appears in any pivotal file path of the other.
(3) ***File names.*** If the sets of pivotal file names between X and Y are identical, then X and Y are placed in the same cluster.
(4) ***Unique functions.*** A unique function is a pivotal function that does not exist in the origin. Projects sharing the same unique function are grouped into the same cluster.

Figure 2 illustrates these characteristics, which effectively cluster candidates with potential reuse relationships. Fork relationships and file paths are straightforward and reliable signals. Although file names alone may not indicate reuse, identical pivotal file names suggest that projects reused the same file set from the origin. Finally, a shared unique function suggests a reuse relationship, as it indicates that both projects inherited the same modification or addition to the origin. If a pair satisfies any of the four criteria, the two projects are placed in the same cluster.

3.3 Reuse Inference (P2)

In P2, Ganadi infers reuse relationships among candidates within each cluster, in three steps: (1) measuring **code similarity** between two projects based on shared pivotal functions, (2) inferring the **reuse direction** using explicit and implicit information, and (3) aggregating the results to determine the overall **reuse relationship**.

3.3.1 Code similarity measurement. Let X and Y be two software projects within the same cluster. Ganadi computes not only the code similarity between X and Y, but also between the origin O and each of X and Y. This allows Ganadi to differentiate three possible relationships: (1) X is reused in Y ($X \rightarrow Y$); (2) Y is reused in X ($Y \rightarrow X$); and (3) Both independently reuse O ($O \rightarrow X$ and $O \rightarrow Y$).

In addition, Ganadi computes similarity by considering only pivotal functions to focus on code regions derived from the origin. Ganadi represents each project as a binary vector based on the presence or absence of pivotal functions, and computes similarity between the resulting embeddings. To this end, Ganadi normalizes each pivotal function (see Section 3.2.2) and hashes it (*e.g.*, `SHA256`).

Given two projects X and Y, F_X and F_Y denote their respective sets of pivotal functions. Let f_i represent a hashed function. We define the union of these functions as the reference function set F:

$$F = F_X \cup F_Y = \{f_1, f_2, \ldots, f_m\}$$

Each software is then embedded as a binary vector of dimension m (let $\mathbf{v}_X$ and $\mathbf{v}_Y$), where the presence of function f_i in project X is encoded as follows.

$$(\mathbf{v}_X)_i = \mathbb{I}(f_i \in F_X) = \begin{cases} 1 & \text{if } f_i \in F_X, \\ 0 & \text{otherwise.} \end{cases}$$

Ganadi then measures the similarity using *cosine similarity* (ϕ). The cosine similarity between two vectors is calculated as follows.

$$\phi(\mathbf{v}_X, \mathbf{v}_Y) = \frac{\mathbf{v}_X \cdot \mathbf{v}_Y}{||\mathbf{v}_X||\,||\mathbf{v}_Y||}$$

Ganadi focuses only on syntactically identical functions in X and Y, based on the assumption that exact matches suffice for identifying reuse: in practice, when projects reuse code from a common origin, a substantial portion of functions remain unmodified [35], providing reliable evidence for inferring reuse direction.

Using this approach, Ganadi computes code similarity for every software pair (X, Y) within a cluster, including the similarity between (1) O and X, (2) O and Y, and (3) X and Y. An important consideration is that the origin O does not have pivotal functions defined. To address this, Ganadi applies a reverse matching strategy: it identifies the functions in O identical to the pivotal functions of X, and designates the corresponding files in O as pivotal files. The similarity is then computed using the same methodology.

3.3.2 Direction inference. Next, Ganadi determines the direction of reuse between two projects X and Y in which a reuse relationship is *assumed* to exist. Note that determining reuse direction fundamentally distinguishes Ganadi from code clone detection: AST-based and token-based clone detectors identify a symmetric relation (whether two fragments are similar) but cannot determine which side is the ancestor. Ganadi resolves this asymmetry by leveraging *explicit* and *implicit* matches.

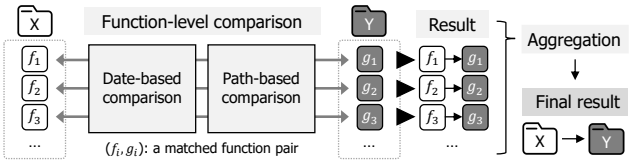


Figure 3: Illustration of implicit matching based on function-level aggregation. GANADI captures fine-grained reuse patterns across functions and infers the reuse direction between software projects by aggregating this evidence.

M1. Explicit matching. First, GANADI focuses on two explicit indicators that can directly confirm reuse relationships: *fork relationships* and *file paths*.

- **Fork relationships.** If Y is a fork of X, GANADI infers a reuse direction from X to Y.
- **File paths.** If the path of a pivotal file in Y contains the name of X, GANADI infers that Y reuses X.

M2. Implicit matching. If directionality cannot be identified through explicit matching, GANADI applies implicit matching using function-level aggregation. This method examines all matched pivotal function pairs (f_i, g_i) where $f_i \in F_X$ and $g_i \in F_Y$, and aggregates evidence from individual functions to determine the overall software-level reuse direction.

For each pair, GANADI employs two comparison methods.

- **Date-based comparison.** GANADI compares the release dates of the earliest versions in which f_i and g_i appeared in X and Y, respectively. The function that appeared earlier is treated as the ancestor.
- **Path-based comparison.** GANADI checks whether one function path fully contains the other. In such cases, the function in the included path is regarded as the reused one, and the other as the ancestor.

Here, version release dates can be inconsistent or even falsified, for example, when repository migrations or re-commits cause a downstream project's metadata to predate the origin. GANADI addresses such unreliable timestamps through three mechanisms. First, explicit matching takes strict precedence over implicit matching; corrupted timestamps thus have no effect when explicit evidence exists. Second, path-based comparison provides a complementary signal even when date-based comparison is misleading. Third, GANADI aggregates directional evidence across all matched function pairs via majority voting, so that a few corrupted timestamps are outvoted by consistent evidence from the remaining functions. Directionality inference fails only when explicit evidence, path signals, and consistent date evidence are all absent.

Figure 3 illustrates the function-level aggregation at a high level. After each comparison, a directional decision is made (*e.g.*, $f_i \rightarrow g_i$). If no clear direction can be determined (*e.g.*, identical release dates), the pair is skipped. The overall reuse direction between projects is inferred based on majority voting across all matched function pairs: GANADI infers $X \rightarrow Y$ if more than half of the function pairs indicate $f_i \rightarrow g_i$.

If no direction is identified from *any* of the functions, GANADI skips inferring a reuse relationship. Although this may suggest a fork relationship, the absence of explicit evidence from earlier matching prevents a confident decision.

3.4 Graph Construction (P3)

Finally, GANADI constructs a reuse genealogy graph.

- **OSS reuse genealogy graph**

  This is a directed acyclic graph that captures reuse relationships between software projects. An OSS reuse genealogy graph is defined as $G = (V, E)$, where V is the set of nodes (software) and E is the set of directed edges ($E \subseteq V \times V$). An edge $e = (v_1, v_2)$ indicates that v_1 is reused in v_2 ($v_1 \rightarrow v_2$).

Even with pairwise directions inferred, naively connecting all edges yields a noisy graph with redundant and conflicting paths, and no prior approach addresses how to consolidate pairwise relationships into a coherent genealogy. GANADI resolves this with priority-based construction rules grounded in how code actually propagates in the OSS ecosystem (*e.g.*, one-to-many reuse).

Specifically, the graph construction follows a principle of evidence strength and directness: explicit evidence takes precedence over implicit evidence, and among implicit connections, GANADI prioritizes the strongest relationships to minimize noise.

Prioritizing explicit evidence. Edges identified through explicit matching (fork or file paths) are always included regardless of similarity scores, as they provide definitive proof of reuse relationships.

Filtering implicit connections. For relationships inferred through implicit matching, GANADI applies systematic filtering. The priorities are assigned in descending order: R1, R2, R3, and R4.

R1. Minimum similarity threshold. Projects with low similarity scores ($\phi(X, Y) < \tau$, where τ is the threshold, *e.g.*, 0.5) are not connected, as weak similarity is unreliable evidence of reuse.

R2. Strongest connection selection. For each project, GANADI connects only to the project with the highest similarity score. Because clear relationships are captured through explicit connections, focusing on the strongest implicit relationship avoids introducing noisy edges. When multiple candidates share the maximum score, all are considered.

R3. One-to-many propagation model. GANADI models realistic code propagation patterns: one OSS project is frequently reused by multiple projects (one-to-many), but it is rare for developers to gather identical OSS code fragments from multiple projects and merge them into one (many-to-one). Therefore, when multiple edges point to the same project, only the strongest edge (*i.e.*, highest similarity or explicit connection) is retained.

R4. Direct origin preference. Finally, when $\phi(O, Y)$ is greater than $\phi(X, Y)$, this implies that Y was more directly derived from O rather than through intermediate X. Hence, GANADI removes the $X \rightarrow Y$ edge to preserve the most direct genealogy path.

Table 1: A partial result of similarity and direction inference within a cluster for `zlib` (the origin O).

X	Y	$\phi(X,Y)$	$\phi(X,O)$	$\phi(O,Y)$	Direction
FreeType	Godot	0.3543	0.3493	0.6643	$X \rightarrow Y$ (Explicit)
FreeType	SumatraPDF	0.1026	0.3493	0.5692	$X \rightarrow Y$ (Explicit)
FreeType	Redot	0.3112	0.3493	0.4329	$X \rightarrow Y$ (Implicit)
FreeType	Miniblink49	0.1071	0.3493	0.5036	$X \rightarrow Y$ (Explicit)
Godot	Redot	0.6282	0.6643	0.4329	$X \rightarrow Y$ (Explicit)
Godot	SumatraPDF	0.4986	0.6643	0.5692	$X \rightarrow Y$ (Implicit)
Godot	Miniblink49	0.5346	0.6643	0.5036	$X \rightarrow Y$ (Implicit)
SumatraPDF	Redot	0.1751	0.5692	0.4329	$X \rightarrow Y$ (Implicit)
SumatraPDF	Miniblink49	0.5674	0.5692	0.5036	$Y \rightarrow X$ (Implicit)
Redot	Miniblink49	0.1771	0.4329	0.5036	$Y \rightarrow X$ (Implicit)

* All software versions are the latest as of June 2025.

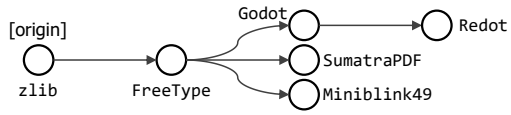


Figure 4: OSS reuse genealogy graph generated from Table 1.

As an example, Table 1 shows a subset of P2 results for `zlib`. Based on *explicit* evidence, four edges are constructed.

> e_1. `FreeType` → `Godot` (explicit)
> e_2. `FreeType` → `SumatraPDF` (explicit)
> e_3. `FreeType` → `Miniblink49` (explicit)
> e_4. `Godot` → `Redot` (explicit)

Assume that $\tau = 0.5$. Excluding the edges created by explicit evidence, any edge with $\phi(X,Y) < \tau$ is disregarded according to **R1**. As a result, only the two edges remain among all implicit edges.

e_5. `Godot` → `Miniblink49` (edge candidate)
e_6. `Miniblink49` → `SumatraPDF` (edge candidate)

Next, among the two candidate edges (e_5 and e_6), only e_6 satisfies **R2** and is retained, while e_5 is removed, as the similarity from `Godot` to `Redot` is higher than that from `Godot` to `Miniblink49`. Also, because an explicit edge to `SumatraPDF` (e_2) already exists, e_5 should be removed (**R3**). Although this case is not explicitly shown in the example, even if the similarity between `Godot` and `SumatraPDF` were 0.5, the edge would still not be established due to rule **R4**, as $\phi(O,Y)$ (0.5692) is higher than $\phi(X,Y)$ (0.5). Consequently, **only the edges e_1 through e_4 remain**.

Through this process, the reuse genealogy among the projects within each cluster is determined. As a final step, Ganadi connects the origin to all nodes with zero indegree, ensuring that every project can be traced back to the origin. This includes both software projects with no direct reuse relationships and starting points of existing reuse chains. Individual software projects that do not belong to any cluster are also connected to the origin, allowing Ganadi to produce a comprehensive reuse genealogy.

Figure 4 shows the graph generated based on Table 1. Ganadi successfully reconstructs the reuse genealogy of `zlib`: `FreeType` reuses code from `zlib`; `Godot`, `SumatraPDF`, and `Miniblink49` inherit `zlib` component code through `FreeType`; and `Redot` inherits `zlib` from `Godot`, as it was developed as a fork of `Godot`.

4 Evaluation

In this section, we experimentally evaluate Ganadi to answer the following three research questions.

RQ1. Accuracy. How precisely and effectively does Ganadi identify OSS reuse genealogy? (Section 4.1)

RQ2. Performance and scalability. How efficient and scalable is Ganadi? (Section 4.2)

RQ3. Application. How does Ganadi contribute to securing the software supply chain? (Section 4.3)

We ran Ganadi on an AWS m7i.4xlarge instance running Ubuntu 22.04.5 LTS, equipped with an Intel Xeon Platinum 8488C Processor (16 vCPUs, 2.4GHz), 64GB RAM, and a 2TB NVMe SSD.

Implementation of Ganadi. Ganadi comprises approximately 1,300 lines of Python code, excluding external libraries such as `Tree-sitter` and `scikit-learn` [26]. It includes three modules: a *dataset collector* that builds the software pool, a *clusterer* that identifies and groups candidates (see Section 3.2), and a *reuse genealogy identifier* that infers reuse directions and constructs the genealogy graph (Section 3.3 and Section 3.4).

Software pool construction. To build the software pool, Ganadi collected all versions of the top 2,500 C/C++ repositories on GitHub ranked by stargazers (as of June 2025), including prominent projects such as `Linux Kernel`, `Wireshark`, and `Redis`. Ganadi then extracted all functions from each repository using `Tree-sitter`. Following prior work (*e.g.*, [2, 35]), we consider versions based on `GitHub` tags, but only include those that follow `major.minor.patch` semantic versioning. Release dates were retrieved using `Git` commands (*e.g.*, `git tag --sort=creatordate`). Repositories without tags or non-semantic versioning were excluded due to the lack of reliable release date information. Consequently, Ganadi stored codebases and metadata for 2,006 repositories in the pool.

4.1 Accuracy of Ganadi

We first evaluate the genealogy construction accuracy of Ganadi on widely reused OSS projects.

4.1.1 Target software selection. To avoid bias in target OSS selection, we consider the following three criteria: (1) the OSS should be popular and thus well-maintained, (2) the selected OSS projects should collectively provide at least 1,000 propagation paths within the pool for reliable accuracy evaluation, and (3) the target OSS projects should span diverse domains.

To this end, we referred to the results of Centris [35], which analyzed OSS components for 15,000 popular projects. We filtered these results to identify prime OSS (*i.e.*, projects that do not incorporate other components) and sorted them by reuse frequency. Here, the top 20 OSS projects collectively provided over 1,000 reuse paths, satisfying the second criterion; we thus selected them as evaluation targets (see Table 2). This set includes popular OSS such as `Lua`, `Libxml2`, and `json-c`, and spans diverse domains including compression, file systems, and media processing.

4.1.2 Accuracy comparison with Cneps. We first measure and compare the accuracy of reuse genealogy identification between Ganadi and Cneps [23], which analyzes component dependencies.

Adaptation of CNEPS for evaluation. Although CNEPS does not directly identify OSS reuse genealogies, it infers the dependency direction among reused components. For example, given `Godot` as input (see Figure 4), CNEPS identifies that `FreeType` and `zlib` are reused, yielding `Godot` ⟶ `Freetype` ⟶ `zlib`.

Therefore, we construct a reverse dependency graph from the CNEPS results. We invert the direction of each dependency edge so that the component that originally appears at the end of the dependency chain becomes the starting point of the graph, as follows: `Godot` ⟵ `Freetype` ⟵ `zlib` (reverse CNEPS results). This yields a structure that approximates the reuse propagation relationships.

We initially executed CNEPS on the 2,006 projects in our pool. However, when queried with a downstream project X, CNEPS sometimes failed to reveal that X reuses the target OSS. In such cases, the relationship might be recoverable indirectly: querying a project Y (a downstream of X outside our pool) returned the chain OSS$\rightarrow X \rightarrow Y$, from which the missing relationship could be extracted. We therefore executed CNEPS on approximately 10,000 projects from the CNEPS dataset to maximize coverage, and retained only the paths within our pool (*e.g.*, discarding the $X \rightarrow Y$ edge for Y outside the pool), ensuring a fair comparison. For each project, we used the version released as of June 2025, as analyzing multiple versions increases computational cost while yielding similar results.

Methodology. We assessed the accuracy of GANADI and CNEPS by analyzing the reuse directions in the genealogy graphs. However, validation remains challenging due to the absence of benchmark datasets for C/C++, where reuse relationships are rarely documented. Therefore, we manually analyzed all the results. The evaluation is conducted by two experts: one with over 15 years and the other with over five years of experience in software engineering and security. We primarily rely on pivotal file paths or metadata (*e.g.*, `README`) that provide clues about the origin. We then analyze source code and comments, as well as the commit history, to assess the accuracy of reuse relationships as thoroughly as possible. The decision is reached through discussion between the two evaluators.

We consider five evaluation metrics: *true positives* (`TP`), *false positives* (`FP`), *false negatives* (`FN`), *precision* (`#TP/(#TP + #FP)`), and *recall* (`#TP/(#TP + #FN)`). A TP is a correctly identified reuse relationship, an FP is an incorrect inference, and an FN is a missed reuse relationship. *True negatives* are excluded, because the number of non-reused projects is overwhelmingly large, potentially introducing bias. We set the threshold θ (see Section 3.2.2) for candidate identification to the minimum value (*i.e.*, projects are considered candidates if they share at least one function with the target OSS), and set the threshold τ (see Section 3.4) for reuse relationship analysis to 0.5. Experiments on threshold sensitivity are presented in Section 4.1.5.

Overall results. Table 2 shows the results. More than 1,600 correct reuse relationships were identified; over 20% involved propagation through at least one intermediate project, with the longest chain passing through four *intermediate* projects (Table 3). Despite these challenging transitive cases, GANADI achieved 84.85% precision and 95.76% recall, whereas CNEPS achieved higher precision (90.73%) but considerably lower recall (23.21%).

Result analysis: CNEPS. Although CNEPS generated few FPs, it yielded many FNs (23.21% recall). CNEPS reported fewer FPs because it identifies OSS components based on CENTRIS and traces clone paths within files to filter out incidental code similarity, thereby considering only clear and verifiable reuse cases. However, this design also causes the FNs: CNEPS focuses on one-depth dependencies rather than transitive paths, and emphasizes only explicit reuse patterns (*e.g.*, `#include`), overlooking many indirect or implicit reuse relationships. In particular, CNEPS failed to identify reuse of `json-c`, `stb`, and `libsodium` as components.

Table 2: Accuracy measurement results of CNEPS and GANADI on 20 widely reused OSS projects (P: precision, R: recall).

Target OSS	CNEPS [23]					GANADI				
	#TP	#FP	#FN	P (%)	R (%)	#TP	#FP	#FN	P (%)	R (%)
json-c	2	0	607	100.00	0.33	594	49	15	92.38	97.54
stb	21	2	146	91.30	12.57	160	64	7	71.43	95.81
zlib	146	12	15	92.41	90.68	148	39	13	79.14	91.93
libsodium	3	0	123	100.00	2.38	123	24	3	83.67	97.62
libuv	7	1	89	87.50	7.29	94	9	2	91.26	97.92
xxHash	69	6	9	92.00	88.46	75	16	3	82.42	96.15
Lua	60	7	17	89.55	77.92	68	25	9	73.12	88.31
Expat	20	2	17	90.91	54.05	35	5	2	87.50	94.59
flex	4	1	28	80.00	12.50	29	8	3	78.38	90.63
PCRE2	0	0	29	0.00	0.00	27	6	2	81.82	93.10
libunwind	3	0	24	100.00	11.11	26	1	1	96.30	96.30
Ogg	0	0	24	0.00	0.00	23	3	1	88.46	95.83
Libxml2	5	0	16	100.00	23.81	20	3	1	86.96	95.24
Vorbis	0	0	22	0.00	0.00	19	11	3	63.33	86.36
TZ	15	1	6	93.75	71.43	19	2	2	90.48	90.48
libzip	3	0	16	100.00	15.79	19	1	0	95.00	100.00
libpcap	2	2	15	50.00	11.76	16	6	1	72.73	94.12
libCoAP	1	0	14	100.00	6.67	15	2	0	88.24	100.00
LuaBitOP	11	4	3	73.33	78.57	14	0	0	100.00	100.00
libui	0	0	11	0.00	0.00	11	0	0	100.00	100.00
Total	372	**38**	1,231	**90.73**	23.21	**1,535**	274	**68**	84.85	**95.76**

Table 3: Distribution of propagation depths of reuse relationships identified by CNEPS and GANADI.

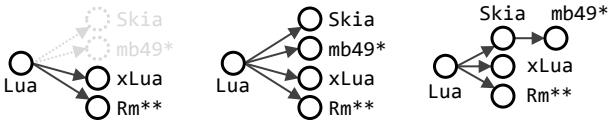
Tool	Propagation depth					Total
	2	3	4	5	6	
CNEPS	298	68	5	1	0	372
GANADI	1,169	354	8	3	1	1,535

Result analysis: GANADI. GANADI successfully identified even transitive OSS reuse. For example, in the case of `stb`, we found that `Filament` reuses `stb` both directly and indirectly through `glfw` (`stb` ⟶ `glfw` ⟶ `Filament` and `stb` ⟶ `Filament`). In particular, `Filament` includes `stb` code in its own source tree and through `glfw` (`/third_party/glfw/deps/stb_image_write.h`).

However, some FPs and FNs were observed. FPs arose from three causes: (1) missing intermediate projects, (2) reuse of identical origin versions with similar path structures, and (3) commonly shared code snippets (*e.g.*, cryptographic code). For the first case, if Y is missing from the pool in a chain $X \rightarrow Y \rightarrow Z$, an incorrect relationship such as $X \rightarrow Z$ may be inferred. The latter two cases occur when unrelated projects reused the same origin version with similar path structures, or when only short, generic code was shared across projects. Most FNs occurred when the reuse relationship was implicit and the proportion of shared functions fell below τ.

Table 4: Accuracy comparison (GANADI *vs.* V0FINDER).

Target OSS	V0FINDER [34]					GANADI				
	#TP	#FP	#FN	P(%)	R(%)	#TP	#FP	#FN	P(%)	R(%)
Lua	4	7	73	36.36	5.19	68	25	9	73.12	88.31
Expat	17	20	20	45.95	45.95	35	5	2	87.50	94.59
Ogg	4	0	20	100.00	16.67	23	3	1	88.46	95.83
Libxml2	4	3	17	57.14	19.05	20	3	1	86.96	95.24
Vorbis	10	0	12	100.00	45.45	19	11	3	63.33	86.36
Total	39	30	142	56.52	21.55	165	47	16	**77.83**	**91.16**



(a) V0FINDER (b) CNEPS (c) GANADI

Figure 5: Key differences among V0FINDER, CNEPS, and GANADI (mb49: `miniblink49`, Rm: `Rainmeter`).

4.1.3 Accuracy comparison with V0FINDER. We next compare the accuracy of GANADI with that of V0FINDER [34]. Although identifying reuse relationships is not the primary objective of V0FINDER, it relies on reuse information to detect vulnerability propagation.

Methodology. Unlike CNEPS, V0FINDER is limited to projects with known vulnerabilities. We thus restricted our evaluation to projects that satisfy its requirements: (1) a reported CVE exists, and (2) the corresponding patch is available as a `GitHub` commit. As a result, five OSS projects were selected.

Result analysis. Table 4 presents the accuracy comparison results. The most significant difference lies in recall: GANADI achieved 91.16%, while V0FINDER reached only 21.55%. This gap is caused by two main limitations of V0FINDER: it (1) detects reuse only when vulnerable code is propagated and (2) infers reuse based on coarse signals (*e.g.*, overall code similarity). Although V0FINDER produced fewer FPs (30 *vs.* GANADI's 47), this is primarily due to its overly strict criteria for identifying reuse edges. Supporting this, it identified only 39 TPs, significantly fewer than GANADI (165 TPs).

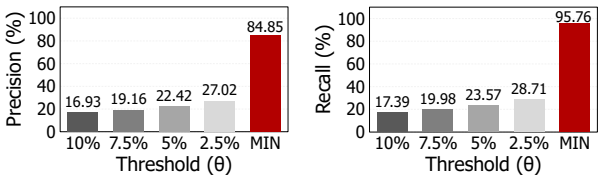
Overall, CNEPS identifies origins but lacks accuracy for detailed reuse relationships, while V0FINDER is limited to cases where vulnerabilities have propagated. Figure 5 illustrates these differences.

4.1.4 Ablation study. To quantify the contribution of each component, we removed one component at a time across the three phases and measured the accuracy on the 20 target OSS projects. Specifically, when a clustering criterion is removed, candidate pairs are no longer grouped by that criterion; when explicit (or implicit) direction matching is removed, directions are inferred using only the remaining method; and when explicit (or implicit) edges are removed, the graph is constructed using only the other type.

Table 5 presents the results. Removing any single component degraded both precision and recall, indicating that every component is necessary. The largest degradation occurred when explicit edges were removed from graph construction (58.39% precision and 52.96% recall): relationships evidenced by forks or file paths often exhibit low code similarity, and without explicit prioritization, they were discarded by the similarity threshold τ. Conversely, removing

Table 5: Ablation study results. Each row removes one component from GANADI.

Phase	Removed component	#TP	#FP	#FN	P(%)	R(%)
GANADI (all components)		**1,535**	**274**	**68**	**84.85**	**95.76**
Clustering (P1)	Fork relationships	953	511	650	65.10	59.45
	File paths	977	514	626	65.53	60.95
	File names	962	511	641	65.31	60.01
	Unique functions	1,145	385	458	74.84	71.43
Direction (P2)	Explicit matching	963	503	640	65.69	60.07
	Implicit matching	1,098	391	505	73.74	68.50
Graph (P3)	Explicit edges	849	605	754	58.39	52.96
	Implicit edges	1,077	363	526	74.79	67.19



(a) Precision (b) Recall

Figure 6: Effect of θ on precision and recall.

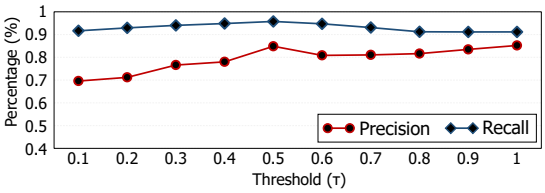


Figure 7: Effect of τ on precision and recall.

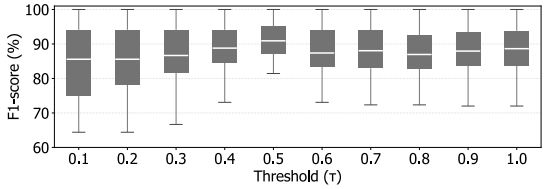
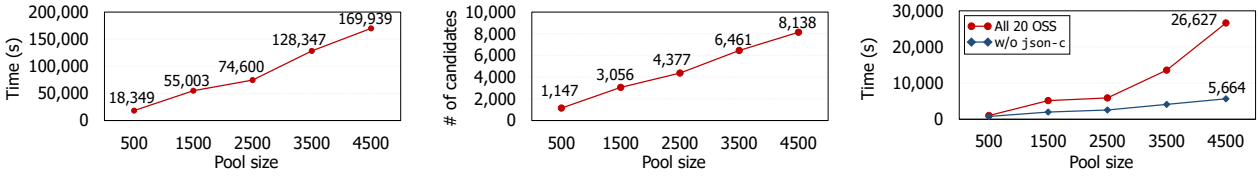


Figure 8: Effect of τ on F1-score.

implicit edges lowered recall to 67.19%, as only the relationships confirmed by explicit evidence survived; it also introduced FPs, because when an intermediate edge is lost (*e.g.*, $B \rightarrow C$ in a chain $A \rightarrow B \rightarrow C$), the disconnected project is directly attached to the origin, creating an incorrect edge ($A \rightarrow C$). A similar pattern was observed in direction inference: removing explicit matching degraded accuracy more than removing implicit matching, consistent with the precedence of explicit evidence in our design. Among the clustering criteria, fork relationships, file paths, and file names contributed comparably, while unique functions contributed less but still meaningfully.

4.1.5 Threshold sensitivity. To evaluate the impact of θ (Section 3.2), we varied it from the minimum setting (*i.e.*, at least one mapped function) to 2.5%, 5%, 7.5%, and 10%; We limited the maximum value to 10%, following CENTRIS, as higher values yielded too few candidates. As shown in Figure 6, both precision and recall dropped significantly as θ increased. Although a higher θ was expected to



(a) Pool construction time **(b) # of candidates** **(c) Genealogy identification time**

Figure 9: Scalability of Ganadi with varying pool sizes: (a) pool construction, (b) identified candidates, and (c) identification time for the 20 target projects (Table 2). Even as the pool grows, all measurements increase near-linearly except for `json-c`.

reduce FPs by narrowing the candidate set, it instead increased both FPs (due to missing intermediate projects) and FNs (due to missed TPs). Hence, Ganadi adopts the minimum θ as the default.

To assess the impact of τ (Section 3.4), we fixed θ at its minimum value and varied τ from 0.1 to 1.0 in increments of 0.1. Figure 7 shows the precision and recall aggregated over the 20 target OSS projects, and Figure 8 shows the distribution of F1-scores across individual origins. As shown in Figure 7, precision consistently improved as τ increased, as higher thresholds filter out weak similarity edges. Recall, however, peaked at $\tau = 0.5$ and degraded in both directions: higher τ values discarded genuine reuse relationships, while lower values admitted noisy edges that displaced correct ones during graph construction, increasing FNs as well as FPs. Figure 8 shows a trend at the F1 level: $\tau = 0.5$ achieved the highest average F1-score with relatively low variance, and the F1-scores remained stable across a wide range of τ (0.3 to 1.0), indicating that the accuracy of Ganadi is robust to the choice of τ. Thus, we selected $\tau = 0.5$ as the default. Note that θ and τ are the only tunable parameters of Ganadi; the other design elements (*e.g.*, the clustering criteria and graph construction rules) are rule-based and require no tuning.

4.2 Performance and Scalability

4.2.1 Performance. To evaluate performance, we measured the time required for Ganadi to construct the software pool and to build the OSS reuse genealogies. First, constructing the software pool (Section 3.2.1) took 20.8 hours in our environment. However, this is a one-time preprocessing step and does not impact run-time performance. Next, identifying the reuse genealogy for the 20 target OSS projects (excluding database I/O time) took 314.29 s per project on average, with a median of 86.83 s. The gap between the average and median is caused by `json-c`, which yielded 644 candidates and required approximately one hour to analyze, because the runtime is largely determined by the number of candidates. For all other OSS projects (fewer than 200 candidates), the average execution time was less than one minute. Even under the most aggressive setting (*i.e.*, the minimum θ), Ganadi took only 148.79 s on average per project (excluding outliers), demonstrating its efficiency.

4.2.2 Scalability. To evaluate how Ganadi scales with the software pool, we varied the pool size from 500 to 4,500 repositories and measured, for the 20 target OSS projects, (1) pool construction time, (2) the number of identified candidates, and (3) genealogy identification time. Figure 9 presents the measurement results.

Pool construction time grew linearly with the pool size (9.3 times for a nine-fold larger pool), as every step before candidate identification handles each repository independently.

The total number of candidates grew more slowly than the pool (7.1 times under the nine-fold growth); because only the projects that share functions with the origin become candidates, expanding the pool does not proportionally increase the comparison targets.

Identification time did not exhibit quadratic growth: excluding `json-c`, it grew 8.1 times, closely tracking the number of candidates rather than its quadratic bound, because clustering partitions candidates into smaller groups and confines pairwise comparisons within each cluster. The only exception was `json-c`, whose identification time grew 83.9 times; `json-c` is reused so widely that it yielded far more candidates than any other origin (1,406 at the largest pool, compared to at most 523 for the others), incurring the quadratic pairwise cost. Even in this case, however, the analysis remains tractable, taking approximately one hour in the 2,500-repository pool as a one-time analysis per origin.

4.2.3 Complexity analysis. To examine the scalability of Ganadi, we analyze the algorithmic complexity of each phase.

Let n denote the pool size and c denote the number of candidates for an origin identified by Ganadi. First, pool construction costs $O(n)$, as it processes the functions of each repository exactly once, independently of the others. Candidate identification also costs $O(n)$: it screens the pool by matching the origin's function hashes against each repository's function set, where each check is a constant-time hash lookup. The remaining phases are independent of n and depend only on c: clustering costs $O(c^2)$, as it examines the four clustering criteria for every pair of candidates. Reuse inference costs $O(\sum_i |C_i|^2)$, where C_i denotes the resulting clusters, because pairwise similarity and direction comparisons are confined within each cluster; note that $\sum_i |C_i|^2 \leq c^2$, with equality only when all candidates fall into a single cluster. Graph construction is linear in the number of inferred edges, as rules R1 to R4 require only a constant number of comparisons per edge.

Our scalability experiments (Section 4.2.2) are consistent with this analysis: as the pool grew, the number of candidates grew sublinearly, and the clustering kept the pairwise cost far below its quadratic bound, so that the overall identification time grew roughly in line with the pool size, except for `json-c` described above. For pools beyond our experimental scale, the same behavior is expected: except for such rare origins that are almost universally reused, the overall cost grows near-linearly with the pool, indicating that Ganadi scales to much larger pools.

4.3 Application: Vulnerability Detection

Next, we evaluate how Ganadi can strengthen supply chain security via vulnerability detection.

Table 6: Vulnerability detection results with and without genealogy (432 TPs identified via manual analysis).

Tool	Setting	#TP	#FP	#FN	P(%)	R(%)	F1(%)
FIRE	Baseline	420	177	12	70.35	97.22	81.63
	Genealogy	382	74	50	83.77 (+13.42)	88.43 (-8.79)	86.04 (+4.41)
VUDDY	Baseline	146	93	286	61.09	33.80	43.52
	Genealogy	115	38	317	75.16 (+14.07)	26.62 (-7.18)	39.32 (-4.20)

4.3.1 Methodology. We evaluate genealogy-driven vulnerability detection by applying two existing tools, VUDDY [20] and FIRE [6], under two settings: with and without genealogy information.

In Section 4.1, Ganadi identified the genealogies of 20 OSS projects, which collectively include 957 unique software projects (*i.e.*, downstreams). Using the NVD JSON feed, we collected CVEs that provide patches as `GitHub` commits [32, 33] across these projects, obtaining 1,850 unique CVEs.

Based on this, we conduct the following two experiments:

- **Baseline (no genealogy).** We apply VUDDY and FIRE without leveraging genealogy information by mapping each of the 957 software projects to all 1,850 CVEs previously reported in the collected projects.
- **Genealogy-driven approach.** When analyzing a given software project, we restrict the vulnerability search space using genealogy information by tracing only CVEs previously reported in projects within the same genealogy, with approximately 200 unique CVEs per genealogy on average.

We exclude vulnerabilities reported by software that were included in Ganadi's FPs (see Section 4.1). This exclusion isolates the impact of genealogy accuracy on vulnerability management, as including vulnerabilities from unrelated projects would introduce noise unrelated to genealogy quality. The detection results of both tools were manually analyzed and categorized into TPs and FPs by the two researchers who conducted the accuracy evaluation.

To further leverage genealogy information, we examined the potential FNs of the two tools as follows. First, we selected a target vulnerability from one project within a genealogy. Second, we examined possible propagation in a structured order (downstream, upstream, then other related projects), analyzing the security patch to identify vulnerable functions, defined as those containing code lines removed by the patch [6, 20, 32, 33]. Third, for each project in the genealogy, we verified whether the vulnerable function had been reused and whether the fix had been applied; when the reused code remained unpatched, we attempted to trigger the vulnerability using the original Proof-of-Concept when available.

4.3.2 Result analysis. Table 6 presents the results. The baseline identified more TPs, however, this gap is not a fundamental limitation; it is a direct consequence of Ganadi's genealogy FNs. Because the genealogy-driven approach restricts its search space to projects within a given genealogy, vulnerabilities originating from missed downstreams fall outside its search space, and are detectable only by the exhaustive baseline.

Despite this, the genealogy-driven approach demonstrates three notable strengths. First, it **improves precision** by restricting the analysis scope to evolutionarily and structurally related projects.

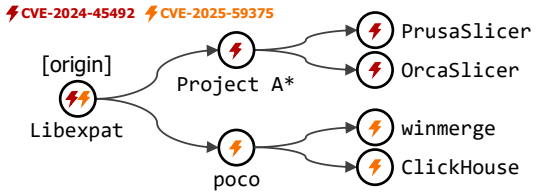


Figure 10: OSS reuse genealogy graph for `Libexpat`, showing two vulnerabilities propagating along distinct paths. We reported them to all repositories in the graph. Project A remains unpatched and is anonymized to prevent misuse.

In this setting, FIRE's precision increases from 70.35% to 83.77%, and VUDDY's from 61.09% to 75.16%. This reduction in FPs lowers validation overhead and improves practical usability.

Second, the genealogy-driven approach achieves comparable detection performance while **relying on a much smaller CVE dataset**. It uses only 11% on average (200 per genealogy *vs.* 1,850), yet the F1-score increases by 4.41% for FIRE and decreases by only 4.2% for VUDDY, indicating that similar detection performance can be maintained without an exhaustive vulnerability dataset while reducing analysis overhead.

Last, the genealogy information provides explicit propagation paths across related software projects, **offering structural context on how vulnerabilities are inherited**. This enables more systematic manual validation and investigation. In practice, using the aforementioned manual procedure, two experts analyzed vulnerabilities across 20 genealogies in under one hour and identified 12 vulnerabilities that were not detected by FIRE and VUDDY.

4.3.3 Responsible disclosure. During the experiment, we identified 48 triggerable, unpatched propagated vulnerabilities, including 12 vulnerabilities manually detected. We reported all vulnerabilities to the respective teams or maintainers. As of March 2026, 23 vulnerabilities have been patched, including those in widely used software such as `F-Stack`, `Tendis`, and `Wireshark`. One CVE has been assigned for our reported findings (CVE-2025-26269 in `Dragonfly`).

Case study. `Libexpat` reported CVE-2024-45492 (an integer overflow vulnerability; severity: Critical) in 2024 and CVE-2025-59375 (an unbounded resource allocation vulnerability; severity: High) in 2025. Upon examining the genealogy of `Libexpat`, we identified previously undiscovered vulnerability propagation paths, as illustrated in Figure 10. We reported these findings, which were subsequently patched. In particular, downstream projects such as `WinMerge` and `ClickHouse`, which incorporate `Libexpat` via `poco`, made it difficult to analyze the propagation of these vulnerabilities through conventional means. However, genealogy-based vulnerability management enabled systematic tracing of the propagation paths, demonstrating its effectiveness in uncovering previously unidentified vulnerabilities.

5 Discussion

Threats to validity. In the experiment, we collected 2,500 repositories to build a pool and analyzed the reuse genealogy of 20 target OSS. The results identified over 1,600 reuse relationships; however,

this remains insufficient to reflect the overall software environment. In addition, due to the absence of ground truth, we manually analyzed Ganadi's results. The two validators independently agreed on 98% of the results. For the remaining 2%, they reviewed the relevant evidence and resolved their disagreements through discussion. Because the senior validator's judgment was adopted in most of these cases, the resulting labels may still contain human error or subjective bias. Moreover, due to the absence of existing research with nearly the same purpose as ours, we indirectly compared Ganadi with Cneps and V0Finder. We do not intend to diminish the existing studies, but rather to demonstrate that Ganadi's algorithm is effective for reuse genealogy analysis.

Limitations and future work. First, Ganadi operates in environments where C/C++ source code is available. Next, Ganadi may fail to infer reuse directionality when explicit evidence is unavailable. When neither fork relationships nor file-path signals exist, Ganadi falls back on implicit matching, which relies on code birth dates. If timestamps are inconsistent, date-based comparison may yield incorrect directions, and when path-based comparison also provides no signal, directionality inference may fail entirely. Although explicit matching takes strict precedence and mitigates most such cases, we acknowledge this residual limitation. We plan to mitigate it by considering additional features (*e.g.*, incorporating natural language processing into `README` files). Moreover, there are reuse relationships that cannot be identified by the pivotal function-based approach. We plan to consider incorporating structural information and various metadata files. Finally, Ganadi focuses only on analyzing reuse genealogy, while vulnerability identification requires the use of other tools or human analysis. We plan to devise a framework that can perform comprehensive vulnerability management.

Language extensibility. Although Ganadi targets C/C++, its methodology is largely language-agnostic, as it relies on function-level code and metadata that are available in most programming languages. Nevertheless, applying Ganadi to other ecosystems would require accounting for their ecosystem-specific characteristics. For example, in ecosystems with well-established package managers (*e.g.*, Java), code reuse primarily occurs through declared dependencies, leaving fewer copy-based traces to analyze. At the same time, these ecosystems often provide richer explicit metadata, such as dependency manifests, which could be incorporated into Ganadi to improve genealogy accuracy.

6 Related Work

Software composition analysis (SCA). Several approaches attempt to identify reused third-party libraries in software codebases (*e.g.*, [13, 19, 23, 38, 40]). For example, some approaches (*e.g.*, [14, 35, 37]) identify OSS components by adopting Locality Sensitive Hashing algorithms at the function level. Cneps [23] analyzes dependencies between components based on function call-based modules. BinaryAI [13] identifies OSS in binary files by combining a transformer model and link-time locality. However, these approaches can only identify the origins, without capturing the ancestor–descendant relationships in the OSS genealogy.

Origin and code evolution analysis. Some studies have analyzed the origin and evolution of source code (*e.g.*, [8, 9, 11, 12, 16, 18, 29]). Inoue *et al.* [11] analyzed the genealogy of code clones focused on tracking clones within a single project, while Steidl *et al.* [29] provided methods for tracking code history during project version updates. However, neither study addressed relationships across different projects. Kanda *et al.* [16] used the longest common subsequence to compare the number of similar files and built a product evolution tree, but their analysis failed to capture propagation direction or change history. Hata *et al.* [9] aimed to uncover clone-and-own reuse relationships across 4,592 projects through n-gram similarity, but failed to closely identify how component reuse propagates. Although most prior work has focused on the evolution within a single project, our approach traces how components propagate across multiple projects throughout the ecosystem.

Code clone detection. Many studies aim to identify code clones (*e.g.*, [7, 15, 27, 28, 41]), which can be leveraged to identify reuse genealogies. For example, Gitor [28] uses a global code graph, while Yu *et al.* [41] combine pruned ASTs with a Siamese transformer. Although Ganadi builds on code similarity, it differs fundamentally from clone detection: existing clone detection approaches answer a symmetric question (whether two fragments are similar), whereas Ganadi answers an asymmetric one (which project reused code from which, through what path). To this end, Ganadi introduces components absent in clone detection: pivotal function-based clustering scoped to origin-derived regions, direction inference via explicit evidence and majority voting, and priority-based graph construction (R1-R4).

7 Conclusion

As the reuse of OSS becomes widespread and software supply chains become more complex, identifying reuse genealogies is essential from the perspective of supply chain security. We propose Ganadi, an approach that accurately identifies OSS reuse genealogies by leveraging the concepts of pivotal functions and multi-criteria clustering. Ganadi achieved high accuracy even in complex reuse ecosystems and was able to uncover previously unpatched vulnerabilities, demonstrating its practical value in real-world supply chain security. By leveraging the results of Ganadi, developers can perform comprehensive component management and ultimately contribute to building more secure software supply chains.

Data Availability

The artifacts of Ganadi are publicly available on Zenodo [17] (https://doi.org/10.5281/zenodo.21768585) and GitHub (https://github.com/KIMDONGYEON00/GANADI).

Acknowledgments

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.RS-2024-00440780, Development of Automated SBOM and VEX Verification Technologies for Securing Software Supply Chains), ICT Creative Consilience Program (IITP-2026-RS-2020-II201819, 10%), and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2025-00517788, Research on Intelligent SBOM Generation and Automated Vulnerability Analysis through Multi-level Code Analysis).

References

[1] L Jean Camp and Vafa Andalibi. 2021. SBOM Vulnerability Assessment & Corresponding Requirements. *NTIA Response to Notice and Request for Comments on Software Bill of Materials Elements and Considerations* (2021).

[2] Youngjae Choi and Seunghoon Woo. 2025. TIVER: Identifying Adaptive Versions of C/C++ Third-Party Open-Source Components Using a Code Clustering Technique. In *Proceedings of the 47th International Conference on Software Engineering (ICSE). IEEE.* doi:10.1109/ICSE55347.2025.00188

[3] CISA 2025. *2025 Minimum Elements for a Software Bill of Materials (SBOM).* CISA. https://www.ntia.gov/page/software-bill-materials

[4] Massimiliano Di Penta, Daniel M German, Yann-Gaël Guéhéneuc, and Giuliano Antoniol. 2010. An Exploratory Study of the Evolution of Software Licensing. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 1*. 145–154.

[5] Ruian Duan, Ashish Bijlani, Meng Xu, Taesoo Kim, and Wenke Lee. 2017. Identifying Open-Source License Violation and 1-day Security Risk at Large Scale. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2169–2185.

[6] Siyue Feng, Yueming Wu, Wenjie Xue, Sikui Pan, Deqing Zou, Yang Liu, and Hai Jin. 2024. FIRE: Combining Multi-Stage Filtering with Taint Analysis for Scalable Recurring Vulnerability Detection. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1867–1884.

[7] Nils Göde and Rainer Koschke. 2009. Incremental Clone Detection. In *2009 13th European conference on software maintenance and reengineering*. IEEE, 219–228.

[8] Michael W Godfrey and Daniel M German. 2008. The Past, Present, and Future of Software Evolution. In *2008 Frontiers of Software Maintenance*. IEEE, 129–138.

[9] Hideaki Hata and Takashi Ishio. 2022. Software Supply Chain Map: How Reuse Networks Expand. *arXiv preprint arXiv:2204.06531* (2022).

[10] Kaifeng Huang, Chenhao Lu, Yiheng Cao, Bihuan Chen, and Xin Peng. 2024. VMUD: Detecting Recurring Vulnerabilities with Multiple Fixing Functions via Function Selection and Semantic Equivalent Statement Matching. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3958–3972. doi:10.1145/3658644.3690372

[11] Katsuro Inoue, Yusuke Sasaki, Pei Xia, and Yuki Manabe. 2012. Where Does This Code Come From and Where Does It Go?—Integrated Code History Tracker for Open Source Systems. In *2012 34th International Conference on Software Engineering (ICSE)*. IEEE, 331–341.

[12] Mahmoud Jahanshahi, David Reid, and Audris Mockus. 2025. Beyond Dependencies: The Role of Copy-Based Reuse in Open Source Software Development. *ACM Transactions on Software Engineering and Methodology* 34, 8 (2025), 1–49. doi:10.1145/3715907

[13] Ling Jiang, Junwen An, Huihui Huang, Qiyi Tang, Sen Nie, Shi Wu, and Yuqun Zhang. 2024. BinaryAI: Binary Software Composition Analysis via Intelligent Binary Source Code Matching. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3639100

[14] Ling Jiang, Hengchen Yuan, Qiyi Tang, Sen Nie, Shi Wu, and Yuqun Zhang. 2023. Third-Party Library Dependency for Large-Scale SCA in the C/C++ Ecosystem: How Far Are We?. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1383–1395. doi:10.1145/3597926.3598143

[15] Toshihiro Kamiya, Shinji Kusumoto, and Katsuro Inoue. 2002. CCFinder: A Multilinguistic Token-Based Code Clone Detection System for Large Scale Source Code. *IEEE transactions on software engineering* 28, 7 (2002), 654–670.

[16] Tetsuya Kanda, Takashi Ishio, and Katsuro Inoue. 2013. Extraction of Product Evolution Tree from Source Code of Product Variants. In *Proceedings of the 17th International Software Product Line Conference*. 141–150.

[17] Dongyeon Kim, Seunghoon Woo, and Heejo Lee. 2026. Artifact for 'GANADI: Uncovering C/C++ OSS Reuse Genealogies via Pivotal Function-Based Clustering to Enhance Supply Chain Security'. Zenodo. doi:10.5281/zenodo.21768585

[18] Miryung Kim, Vibha Sazawal, David Notkin, and Gail Murphy. 2005. An Empirical Study of Code Clone Genealogies. In *Proceedings of the 10th European software engineering conference held jointly with 13th ACM SIGSOFT international symposium on Foundations of software engineering*. 187–196.

[19] Seojin Kim, Sungmin Park, and Jihyeok Park. 2025. Debun: Detecting Bundled JavaScript Libraries on Web using Property-Order Graphs. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 78–90. doi:10.1109/ASE63991.2025.00015

[20] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery. In *2017 IEEE symposium on security and privacy (SP)*. IEEE, 595–614. doi:10.1109/SP.2017.62

[21] Seongkyeong Kwon, Seunghoon Woo, Gangmo Seong, and Heejo Lee. 2021. OCTOPOCS: Automatic Verification of Propagated Vulnerable Code Using Reformed Proofs of Concept. In *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 174–185.

[22] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022. Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem. In *Proceedings of the 44th International Conference on Software Engineering*. 672–684.

[23] Yoonjong Na, Seunghoon Woo, Joomyeong Lee, and Heejo Lee. 2024. CNEPS: A Precise Approach for Examining Dependencies among Third-Party C/C++ Open-Source Components. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12. doi:10.1145/3597503.3639209

[24] Hoan Anh Nguyen, Anh Tuan Nguyen, Tung Thanh Nguyen, Tien N Nguyen, and Hridesh Rajan. 2013. A Study of Repetitiveness of Code Changes in Software Evolution. In *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 180–190.

[25] Eric O'Donoghue, Brittany Boles, Clemente Izurieta, and Ann Marie Reinhold. 2023. Impacts of Software Bill of Materials (SBOM) Generation on Vulnerability Detection. In *Proceedings of the 2024 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*. 67–76. doi:10.1145/3689944.3696164

[26] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine Learning in Python. *the Journal of machine Learning research* 12 (2011), 2825–2830.

[27] Hitesh Sajnani, Vaibhav Saini, Jeffrey Svajlenko, Chanchal K Roy, and Cristina V Lopes. 2016. SourcererCC: Scaling Code Clone Detection to Big-code. In *Proceedings of the 38th international conference on software engineering*. 1157–1168.

[28] Junjie Shan, Shihan Dou, Yueming Wu, Hairu Wu, and Yang Liu. 2023. Gitor: Scalable Code Clone Detection by Building Global Sample Graph. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 784–795. doi:10.1145/3611643.3616371

[29] Daniela Steidl, Benjamin Hummel, and Elmar Juergens. 2014. Incremental Origin Analysis of Source Code Files. In *Proceedings of the 11th Working Conference on Mining Software Repositories*. 42–51.

[30] Christopher Vendome, Mario Linares-Vásquez, Gabriele Bavota, Massimiliano Di Penta, Daniel M German, and Denys Poshyvanyk. 2015. When and why developers adopt and change software licenses. In *2015 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 31–40.

[31] Laurie Williams, Giacomo Benedetti, Sivana Hamer, Ranindya Paramitha, Imranur Rahman, Mahzabin Tamanna, Greg Tystahl, Nusrat Zahan, Patrick Morrison, Yasemin Acar, et al. 2025. Research Directions in Software Supply Chain Security. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–38.

[32] Seunghoon Woo, Eunjin Choi, Heejo Lee, and Hakjoo Oh. 2023. V1SCAN: Discovering 1-day Vulnerabilities in Reused C/C++ Open-source Software Components Using Code Classification Techniques. In *32nd USENIX Security Symposium (USENIX Security 23)*. 6541–6556.

[33] Seunghoon Woo, Hyunji Hong, Eunjin Choi, and Heejo Lee. 2022. MOVERY: A Precise Approach for Modified Vulnerable Code Clone Discovery from Modified Open-Source Software Components. In *Proceedings of the 31st USENIX Security Symposium (Security)*. 3037–3053.

[34] Seunghoon Woo, Dongwook Lee, Sunghan Park, Heejo Lee, and Sven Dietrich. 2021. V0Finder: Discovering the Correct Origin of Publicly Reported Software Vulnerabilities. In *30th USENIX Security Symposium (USENIX Security 21)*. 3041–3058.

[35] Seunghoon Woo, Sunghan Park, Seulbae Kim, Heejo Lee, and Hakjoo Oh. 2021. CENTRIS: A Precise and Scalable Approach for Identifying Modified Open-Source Software Reuse. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 860–872. doi:10.1109/ICSE43902.2021.00083

[36] Jiaqi Wu, Lingfeng Bao, Xiaohu Yang, Xin Xia, and Xing Hu. 2024. A Large-Scale Empirical Study of Open Source License Usage: Practices and Challenges. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 595–606.

[37] Jiahui Wu, Zhengzi Xu, Wei Tang, Lyuye Zhang, Yueming Wu, Chengyue Liu, Kairan Sun, Lida Zhao, and Yang Liu. 2023. OSSFP: Precise and Scalable C/C++ Third-Party Library Detection using Fingerprinting Functions. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 270–282.

[38] Susheng Wu, Ruisi Wang, Kaifeng Huang, Yiheng Cao, Wenyan Song, Zhuotong Zhou, Yiheng Huang, Bihuan Chen, and Xin Peng. 2024. Vision: Identifying Affected Library Versions for Open Source Software Vulnerabilities. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1447–1459. doi:10.1145/3691620.3695516

[39] Yang Xiao, Bihuan Chen, Chendong Yu, Zhengzi Xu, Zimu Yuan, Feng Li, Binghong Liu, Yang Liu, Wei Huo, Wei Zou, et al. 2020. MVP: Detecting Vulnerabilities using Patch-Enhanced Vulnerability Signatures. In *29th USENIX Security Symposium (USENIX Security 20)*. 1165–1182.

[40] Heedong Yang, Jeongwoo Lee, Hajin Yun, and Seunghoon Woo. 2026. SBridge: Identifying Source-to-Binary Function Similarity via Cross-Domain Control Block Matching. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 1381–1403. doi:10.1145/3797090

[41] Tianchen Yu, Li Yuan, Liannan Lin, and Hongkui He. 2025. A Multiple Representation Transformer with Optimized Abstract Syntax Tree for Efficient Code Clone Detection. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 587–587. doi:10.1109/ICSE55347.2025.00050

Received 2026-03-26; accepted 2026-06-18